

预处理的艺术

前缀和、差分与双指针 · 9题 · C++ & Python 双语对照

NQ113 前缀和试炼之何为前缀和 AcWing 795

输入一个长度为 n 的整数序列。接下来再输入 m 个询问，每个询问输入一对 l, r 。对于每个询问，输出原序列中从第 l 个数到第 r 个数的和。数据范围： $1 \leq l \leq r \leq n$ ， $1 \leq n, m \leq 100000$ ， $-1000 \leq$ 数列中元素的值 ≤ 1000

输入

第一行包含两个整数 n 和 m 。第二行包含 n 个整数，表示整数数列。接下来 m 行，每行包含两个整数 l 和 r ，表示一个询问的区间范围。

输出

共 m 行，每行输出一个询问的结果。

来源

AcWing 795

输入	输出
5 3	3
2 1 3 6 4	6
1 2	10
1 3	
2 4	

提示 [原题链接](#) [参考题解](#) [Y总讲解](#) [Y总代码](#)

C++

```
#include <iostream>

using namespace std;
const int N = 1000007;
int n, m;

int a[N], s[N];

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++)
        scanf("%d", &a[i]);
    s[0] = 0; //刻意初始化s[0]为0，此句可以不写，写也无妨
    for (int i = 1; i <= n; i++)
        s[i] = s[i - 1] + a[i]; //打表法
    //m次询问
    while (m--)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%d\n", s[r] - s[l - 1]); //求区间和，直接从s
        中读取数值
    }

    return 0;
}
```

Python

Python solution pending

NQ114 前缀和试炼之子矩阵的和 AcWing 796

输入一个 n 行 m 列的整数矩阵，再输入 q 个询问，每个询问包含四个整数 x_1, y_1, x_2, y_2 ，表示一个子矩阵的左上角坐标和右下角坐标。对于每个询问输出子矩阵中所有数的和。数据范围： $1 \leq n, m \leq 1000, 1 \leq q \leq 200000, 1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m, -1000 \leq \text{矩阵内元素的值} \leq 1000$

输入

第一行包含三个整数 n, m, q 。接下来 n 行，每行包含 m 个整数，表示整数矩阵。接下来 q 行，每行包含四个整数 x_1, y_1, x_2, y_2 ，表示一组询问。

输出

共 q 行，每行输出一个询问的结果。

来源

AcWing 796

输入

```
3 4 3
1 7 2 4
3 6 2 8
2 1 2 3
1 1 2 2
2 1 3 4
1 3 3 4
```

输出

```
17
27
21
```

提示 Y总讲解 参考题解 Y总代码

C++

```
#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], s[N][N]; //子矩阵

int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    s[0][0] = 0; //刻意初始化s[0]为0，此句可以不写，写也无妨
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j]; //打表法
    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2;
        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        //求子矩阵和，直接从s中读取数值
        printf("%d\n", s[x2][y2] - s[x2][y1 - 1] - s[x1 - 1][y2] + s[x1 - 1][y1 - 1]);
    }
    return 0;
}
```

Python

Python solution pending

NQ115 差分试炼之何为差分 AcWing 797

输入一个长度为 n 的整数序列。接下来输入 m 个操作，每个操作包含三个整数 l, r, c ，表示将序列中 $[l, r]$ 之间的每个数加上 c 。请你输出进行完所有操作后的序列。数据范围： $1 \leq n, m \leq 100000$ ， $1 \leq l \leq r \leq n$ ， $-1000 \leq c \leq 1000$ ， $-1000 \leq$ 整数序列中元素的值 ≤ 1000

输入

第一行包含两个整数 n 和 m 。第二行包含 n 个整数，表示整数序列。接下来 m 行，每行包含三个整数 l, r, c ，表示一个操作。

输出

共一行，包含 n 个整数，表示最终序列。

来源

AcWing 797

输入

```
6 3
1 2 2 1 2 1
1 3 1
3 5 1
1 6 1
```

输出

```
3 4 5 3 4 2
```

提示 原题链接

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int n, m;
int a[N], b[N];

//差分的插入公式
void insert(int l, int r, int c)
{
    b[l] += c;
    b[r + 1] -= c;
}

int main()
{
    cin >> n >> m;
    //读入数组
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    //构造差分数组b
    for (int i = 1; i <= n; i++)
        insert(i, i, a[i]);
    //输出查询结果
    while (m--)
    {
        int l, r, c;
        cin >> l >> r >> c;
        insert(l, r, c);
    }
    //重新复原a数组(前缀和)
    for (int i = 1; i <= n; i++)
        a[i] = a[i - 1] + b[i];
    //输出a
    for (int i = 1; i <= n; i++)
        cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Python

Python solution pending

NQ116 差分试炼之差分矩阵 AcWing 798

输入一个 n 行 m 列的整数矩阵，再输入 q 个操作，每个操作包含五个整数 x_1, y_1, x_2, y_2, c ，其中 (x_1, y_1) 和 (x_2, y_2) 表示一个子矩阵的左上角坐标和右下角坐标。每个操作都要将选中的子矩阵中的每个元素的值加上 c 。请你将进行完所有操作后的矩阵输出。数据范围： $1 \leq n, m \leq 1000, 1 \leq q \leq 100000, 1 \leq x_1 \leq x_2 \leq n, 1 \leq y_1 \leq y_2 \leq m, -1000 \leq c \leq 1000, -1000 \leq$ 矩阵内元素的值 ≤ 1000

输入

第一行包含整数 n, m, q 。接下来 n 行，每行包含 m 个整数，表示整数矩阵。接下来 q 行，每行包含5个整数 x_1, y_1, x_2, y_2, c ，表示一个操作。

输出

共 n 行，每行 m 个整数，表示所有操作进行完毕后的最终矩阵。

来源

AcWing 798

输入

```
3 4 3
1 2 2 1
```

输出

```
2 3 4 1
4 3 4 1
```

```

3 2 2 1          2 2 2 2
1 1 1 1
1 1 2 2 1
1 3 2 3 2
3 1 3 4 1

```

提示 原题链接

C++

```

#include <iostream>
using namespace std;
const int N = 1007;
int n, m, q;
int a[N][N], b[N][N]; //子矩阵
//二维差分核心操作
void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
int main()
{
    //读入矩阵
    scanf("%d%d%d", &n, &m, &q);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            insert(i, j, i, j, a[i][j]); //构造二维差分矩阵

    //q次询问
    while (q--)
    {
        int x1, x2, y1, y2, c;
        scanf("%d%d%d%d%d", &x1, &y1, &x2, &y2, &c);
        //差分核心操作
        insert(x1, y1, x2, y2, c);
    }
    //重建a[i][j],即求二维前缀和
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            a[i][j] = a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1] + b[i][j];
    //输出a[i][j]
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Python

```
# Python solution pending
```

给定一个长度为n的整数序列，请找出最长的不包含重复数字的连续区间，输出它的长度。数据范围：
 $1 \leq n \leq 100000$

输入 第一行包含整数n。第二行包含n个整数（均在0~100000范围内），表示整数序列。

输出 共一行，包含一个整数，表示最长的不包含重复数字的连续子序列的长度。

来源 AcWing 799

输入	输出
5 1 2 2 3 5	3

提示 原题链接

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int num[N],visited[N];

int main(){
    int n;
    cin>>n;
    for (int i=0; i<n; i++ ) cin>>num[i]; //读入n个数
    //双指针扫描
    int res=0; //初始化为最小值
    for (int i=0, j=0; i<n; i++){
        visited[num[i]]++;
        while(j<=i && visited[num[i]]>1){
            visited[num[j]]--;
            j++; //去掉重复数
        }
        res= max(res,i-j+1); //j--i 之间的数字个数
    }
    cout<<res<<endl;
    return 0;
}
```

Python

Python solution pending

NQ118 数组元素的目标和 AcWing 800

给定两个升序排序的有序数组A和B，以及一个目标值x。数组下标从0开始。请你求出满足 $A[i] + B[j] = x$ 的数对(i, j)。数据保证有唯一解。数据范围：数组长度不超过100000。同一数组内元素各不相同。 $1 \leq \text{数组元素} \leq 10^9$

输入 第一行包含三个整数n, m, x，分别表示A的长度，B的长度以及目标值x。第二行包含n个整数，表示数组A。第三行包含m个整数，表示数组B。

输出 共一行，包含两个整数 i 和 j。

来源 AcWing 800

输入	输出

```
4 5 6
1 2 4 7
3 4 6 8 9
```

```
1 1
```

提示 原题链接

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
const int N=100007;
int n,m,x;
int numA[N],numB[N];

int main(){
    //读入n,m,x其中x为目标和
    scanf("%d%d%d",&n,&m,&x);
    for (int i=0; i<n; i++) scanf("%d",&numA[i]);
    for (int j=0; j<m; j++) scanf("%d",&numB[j]);
    //双指针查找和

    for (int i=0, j= m-1; i<n; i++){
        //check(i,j)
        while (j>=0 && numA[i]+numB[j]>x) j--;
        if (numA[i]+numB[j]==x)//判断是否和相等
        {
            printf("%d %d\n",i,j);
            break;
        }
    }
    return 0;
}
```

Python

```
# Python solution pending
```

NQ119 判断子序列 AcWing 2816

给定一个长度为 n 的整数序列 $a_1, a_2, \dots, a_n$ 以及一个长度为 m 的整数序列 $b_1, b_2, \dots, b_m$ 。请你判断 a 序列是否为 b 序列的子序列。子序列指序列的一部分项按原有次序排列而得的序列，例如序列 $\{a_1, a_3, a_5\}$ 是序列 $\{a_1, a_2, a_3, a_4, a_5\}$ 的一个子序列。

输入 第一行包含两个整数 n, m 。第二行包含 n 个整数，表示 $a_1, a_2, \dots, a_n$ 。第三行包含 m 个整数，表示 $b_1, b_2, \dots, b_m$ 。数据范围 $1 \leq n \leq m \leq 10^5$ $-10^9 \leq a_i, b_i \leq 10^9$

输出 如果 a 序列是 b 序列的子序列，输出一行Yes。否则，输出No。

来源 AcWing 2816

输入

```
3 5
1 3 5
1 2 3 4 5
```

输出

```
Yes
```

C++

```
#include <iostream>
#include <cstdio>
using namespace std;
const int N = 1e5+7; // 我的风格
int a[N], b[N];

int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < m; i++) scanf("%d", &b[i]);

    int p = 0; // 指向a数组的指针
    for (int i = 0; i < m; i++)
    {
        if (p < n && a[p] == b[i]) p++;
    }
    // 完全匹配
    if (p == n) puts("Yes");
    else puts("No");

    return 0;
}
```

Python

Python solution pending

NQ120 AcWing NQ059

AcWing NQ059

AcWing NQ059

输入

见原题

输出

见原题

来源

AcWing NQ059

C++

// AcWing NQ059

Python

```

def three_sum(nums, target):
    res = []
    n = len(nums)
    nums.sort()

    for i in range(n):
        # 跳过重复元素
        if i > 0 and nums[i] == nums[i-1]:
            continue

        # 双指针初始化
        j, k = i + 1, n - 1
        while j < k:
            current_sum = nums[i] + nums[j] + nums[k]

            if current_sum == target:
                # 满足x<y<z
                if nums[i] < nums[j] and nums[j] < nums[k]:
                    res.append([nums[i], nums[j], nums[k]])
                # 跳重复
                while j < k and nums[j] == nums[j+1]:
                    j += 1
                while j < k and nums[k] == nums[k-1]:
                    k -= 1
                # 移动指针
                j += 1
                k -= 1
            elif current_sum > target:
                k -= 1 # 和过大, 右指针左移
            else:
                j += 1 # 和过小, 左指针右移

    return res

def main():
    import sys
    input = sys.stdin.read().split()
    idx = 0
    target = int(input[idx])
    idx += 1
    n = int(input[idx])
    idx += 1
    nums = list(map(int, input[idx:idx+n]))
    idx += n

    result = three_sum(nums, target)
    for triplet in result:
        print('{0} {1} {2}'.format(triplet[0], triplet[1], triplet[2]))

if __name__ == "__main__":
    main()

```

NQ121 AcWing NQ070 AcWing NQ070

AcWing NQ070

输入

见原题

输出

见原题

来源

AcWing NQ070

C++

// AcWing NQ070

Python

```

def lower_bound(arr, left, right, target):
    """查找第一个不小于target的元素的索引"""
    while left < right:
        mid = left + (right - left) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left

def upper_bound(arr, left, right, target):
    """查找第一个大于target的元素的索引"""
    while left < right:
        mid = left + (right - left) // 2
        if arr[mid] <= target:
            left = mid + 1
        else:
            right = mid
    return left

def main():
    import sys
    input = sys.stdin.read().split()
    idx = 0

    n = int(input[idx])
    idx += 1

    g = [-1000000000] * (n + 2)
    for i in range(1, n+1):
        g[i] = int(input[idx])
        idx += 1
    g[n+1] = 1000000000

    m = int(input[idx])
    idx += 1

    for _ in range(m):
        t = int(input[idx])
        idx += 1

        l = lower_bound(g, 1, n+1, t)
        r = upper_bound(g, 1, n+1, t)
        if l <= n:
            l -= 1

        if l == n + 1:
            print(g[n])
        elif r == 1:
            print(g[1])
        else:
            if t - g[l] > g[r] - t:
                print(g[r])
            else:
                print(g[l])

```

```
if __name__ == "__main__":  
    main()
```

— 第10章完 · 共9题 —